Seguridad de archivos mapeados con mmap en moon

Sistemas Operativos | Parcial 3: Seguridad de Archivos en Capa Kernel y mmap

Pablo Manjarres

Objetivo y uso

El módulo `kernel/mmap_guard.ko` protege un archivo: niega mapearlo con escritura compartida o con ejecución. El comando `m_fmap` de moon mapea un archivo y muestra cada llamada al sistema.

```
make && make -C kernel
sudo insmod kernel/mmap_guard.ko path=/tmp/moon_secret.txt
./moon
m_fmap /tmp/moon_secret.txt r      # lectura compartida: permitida
m_fmap /tmp/moon_secret.txt w      # escritura compartida: mmap -> -1 EACCES
sh tests/run_mmap_guard_tests.sh   # ataques, registro y medición
```

Los modos son: `r` lectura, `w` escritura compartida, `p` copia privada, `x` ejecución y `u` lectura ampliada con `mprotect()`.

Parte I: preguntas diagnósticas

1. De mmap() al primer fallo de página. `vm_mmap_pgoff()` llama a `security_mmap_file()`. Luego `do_mmap()` crea un `vm_area_struct` con `vm_file` y `vm_ops`, sin crear PTE. En el primer acceso, la MMU no encuentra la PTE y lanza `#PF`. `handle_mm_fault()` valida `vm_flags`. Después, `filemap_fault()` trae la página del page cache e instala la PTE. Desde ahí los accesos no pasan por el kernel, así que el control debe hacerse al crear la VMA.

2. MAP_SHARED frente a MAP_PRIVATE. `MAP_SHARED` agrega `VM_SHARED`: las PTE apuntan a la página del page cache. Las escrituras llegan al disco y otros procesos las ven. `MAP_PRIVATE` usa COW: la primera escritura crea una copia anónima del proceso. Con un descriptor `O_RDONLY`, el kernel quita `VM_MAYWRITE | VM_SHARED` (`mm/mmap.c:464`).

3. Copy-On-Write. La página empieza compartida y sin permiso de escritura. Al escribir, `do_wp_page()` la copia y deja la copia escribible. Si el kernel confunde una página COW con una escribible, la escritura llega al archivo. Eso pasó con Dirty COW (CVE-2016-5195). Se escribía en `/proc/self/mem` mientras otro hilo descartaba la copia con `MADV_DONTNEED`. Así se modificaban archivos de root, como `/etc/passwd`.

4. Tabla de syscalls frente a LSM. Desde 5.7, `kallsyms_lookup_name` no está exportado. Además, CR0.WP está fijado, así que modificar `sys_call_table` es frágil e inseguro. El gancho LSM `mmap_file` (antes `file_mmap`) está en el camino de todas las rutas, pero el LSM debe compilarse con el kernel. La opción elegida usa kretprobes sobre `security_mmap_file()` y `security_file_mprotect()`. Ve lo mismo que un LSM y se carga como módulo.

5. Cifrado transparente. El descifrado debe estar en el sistema de archivos, debajo del page cache, como hace fscrypt en `->read_folio`. Solo quien tiene la clave puede abrir el archivo. El aislamiento de tablas de páginas impide que otros procesos lean la memoria. Contra ataques físicos se usa cifrado de RAM con SME/SEV o TME.

6. Páginas sucias y msync. Escribir por el puntero solo marca sucia la PTE. El writeback, `msync()` o `fsync()` las llevan al disco. Un filtro sobre `write()` no ve esos cambios. Por eso el módulo impide crear el mapeo escribible.

7. eBPF. Sin recompilar, un programa en `kprobe`/`fexit` sobre `security_mmap_file` o el tracepoint `page_fault_user` puede registrar pid, inode y `prot`. BPF LSM (`lsm/mmap_file`, `CONFIG_BPF_LSM=y`) también puede bloquear. El límite es que no ve los accesos después del primer fallo.

8. O_RDONLY con PROT_WRITE y MAP_PRIVATE. Se permite porque la escritura queda en una copia privada y nunca llega al archivo. El riesgo es que el proceso altere su propia vista. Si COW falla, como en Dirty COW, esto se vuelve escritura real.

9. Carreras con MADV_DONTNEED. El parche de Dirty COW agregó `FOLL_COW`. Desde 6.1, `can_follow_write_pte()` (`mm/gup.c:785`) solo permite escribir con `FOLL_FORCE` en páginas anónimas exclusivas. Además, `mmap_lock` y el lock de la tabla de páginas ordenan las dos operaciones. La prueba de carrera confirma que el archivo no cambia.

10. struct page, tablas de páginas y bits. `struct page` indica si una página es del page cache o anónima. La MMU recorre PGD→P4D→PUD→PMD→PTE y verifica los bits: R/W bloquea escrituras, U/S bloquea el acceso a memoria del kernel y NX bloquea la ejecución. Un mapeo sin `PROT_EXEC` tiene NX, por eso el módulo niega

PROT_EXEC.

Parte II: estrategia con IA

Usé la IA para comparar puntos de intercepción, escribir el primer borrador del módulo y proponer ataques. Todo lo que dijo sobre el kernel lo verifiqué en el código fuente de Linux 6.18.

Contexto	Rol	Restricción	Resultado esperado
Enunciado y repositorio moon	Arquitecto de kernel	Módulo cargable, sin parchear el kernel	Comparar puntos de intercepción
Kernel 6.18 con kretprobes	Programador de módulos	Sin `sys_call_table`, sin CR0, sin advertencias	`mmap_guard.c`
Contrato `ui_trace_*` de moon	Desarrollador del shell	Mismo estilo de los comandos existentes	Comando `m_fmap`
Política del módulo	Pruebas ofensivas	Usuario sin privilegios y verificación del contenido	`mmap_attacks.c` y carrera Dirty COW

Alucinaciones detectadas:

Sugerencia	Por qué falla	Dónde se verificó
Usar `kallsyms_lookup_name()` para hallar `sys_call_table`	No está exportado desde 5.7	`kernel/kallsyms.c`
Quitar CR0.WP con `write_cr0`	El kernel fija y restaura el bit	`arch/x86/kernel/cpu/common.c:422`
Registrar un LSM con `security_add_hooks()` desde el módulo	Es `__init`	`security/security.c:629`
`file_mmap` con firma de 6 argumentos	La firma es de 3.x	`include/linux/lsm_hook_defs.h:202`

Validación: compilé el árbol del kernel en uso con su `/proc/config.gz` y gcc 13. Con gcc 15, `insmod` rechazó el módulo por un `struct module` de otro tamaño. Luego cargué y descargué el módulo, revisé `dmesg` y ejecuté los ataques con y sin módulo.

Arquitectura:

```
Ring 3   mmap(fd, PROT_WRITE, MAP_SHARED)       mprotect(addr, PROT_WRITE)
------------------------------ syscall -------------------------------------
Ring 0   security_mmap_file()   <- kretprobe ->  security_file_mprotect()
             mmap_guard: ¿inode protegido y (escritura compartida o exec)?
             sí -> -EACCES      no -> sigue normal (una comparación)
         do_mmap() -> VMA -> #PF -> filemap_fault() -> PTE   (sin cambios)
```

Opción	Costo	Cobertura	Sin recompilar
`LD_PRELOAD`	Ninguno	Nula: se evita con una syscall directa	Sí
Hook en `sys_call_table`	Bajo	Parcial y bloqueado en 6.x	No
LSM o BPF LSM	Mínimo	Total	No (o requiere `lsm=bpf`)
kretprobe en ganchos LSM	Una comparación por mmap	Igual a LSM	Sí
Instrumentar `handle_mm_fault`	Alto: cada fallo	Solo el primer acceso	Sí

Parte III: implementación y resultados

El módulo resuelve el inode de `path` al cargar. En cada `mmap` o `mprotect` compara ese inode. Si el archivo es el protegido y la petición pide `PROT_EXEC` o escritura compartida, cambia la respuesta a `-EACCES`. Registra cada decisión en `dmesg` y la cuenta en `/sys/module/mmap_guard/parameters/{allowed,denied}`. Debe estar en el kernel porque, después del primer fallo, los accesos por puntero no pasan por ninguna biblioteca. Además, una syscall directa evita `LD_PRELOAD`.

Pruebas (Ubuntu 26.04 en WSL2, kernel 6.18.54.1): pasaron las 18 comprobaciones. Sin el módulo, el mismo usuario sí puede escribir el archivo por mmap. Con el módulo:

- Se permiten la lectura y la copia privada.
- Se rechazan con `EACCES` la escritura compartida, `MAP_SHARED_VALIDATE`, la ejecución y las ampliaciones con `mprotect`.
- En la carrera Dirty COW, el kernel aceptó 153 736 escrituras por `/proc/self/mem`. Todas quedaron en la copia privada y el archivo no cambió.
- `m_fmap w` muestra `mmap -> -1 EACCES`.

Sobrecosto (`mmap` + primer acceso + `munmap`, mediana de 5 corridas de 200000):

Archivo	Sin módulo	Con módulo
Protegido	8560 ns	8445 ns
Otro	8809 ns	8976 ns

La diferencia es menor al 2 % y queda dentro del ruido entre corridas. El número de fallos de página no cambia, porque el módulo no toca `handle_mm_fault`. Medí con `clock_gettime` y `getrusage` porque WSL no trae `perf`.

Limitaciones:

- `write()` sigue dependiendo de los permisos POSIX. Se podría cubrir con un gancho en `security_file_permission`.
- Los mapeos creados antes de cargar el módulo siguen activos.
- Root puede descargar el módulo. Para evitarlo se necesita `lockdown` o BPF LSM desde el arranque.
- El módulo solo protege un archivo.
- No protege contra acceso físico a la RAM o DMA. Para eso se usan IOMMU, TME/SME y SEV-SNP/TDX.